

Plan for Test Implementation

GitHub Repo: <https://github.com/JeremyT21/SQA-Digital-Marketplace>

We built a directory that is representative of the application's complete front end. Within each component of the directory, the corresponding tests of each component will be found. This ensures it is easy to see the tests affecting a code component, since they will be found in the same folder as the code files they validate. This also helps to identify issues with directory structure before implementation. The directory structure conventions are referred from the *Google Developer Documentation Style Guide* (<https://developers.google.com/style>), ensuring standard conventions that are carried out throughout the whole project. For example, the guide outlines that all folder directories use dashes "-" and not underscores "_" as spaces in their name fields, which is what we have done in our directory structure.

The directory structure format is as follows:

SQA-Digital-Marketplace (root directory) > Directory Holding Tests > Feature Family > Required Features > Fail/Success Route > Fail/Success Condition > Input/Output/Description

The following is an example of where you can find the expected input and output (along with applicable expected test files) for a successful or unsuccessful run of the "add-credit-limit" test.

Successful Route - Expected I/O:

input-output-for-tests/account-functionality/add-credit-limit/success-route

Failure Route - Expected I/O:

input-output-for-tests/account-functionality/add-credit-limit/failure-route

In each of these directories are an input.txt file and an output.txt file. In this case there is also a current_user_accounts_file.txt file and a daily_transaction_file.txt file to show how the log files are expected to be once the "add-credit-limit" test has been run either successfully or unsuccessfully.

There are three main categories grouping tests we referred to in the directory structure as "Feature Families". The three feature families were sorted by account functionality, buying functionality, and cart functionality. This covered front end processes for user profiles and account reactions, transactional elements, and components specific to the cart used for purchasing games.

As a clarification for some commonly seen attributes in our input output files, starting_session and final_session being null reflect a user not being signed in.

The test files will be Python files that are run with pytest, and should be run each time a code component is updated or added. These tests use specific input to verify that each test case is able to function properly and give a successful output. For example, given the “add-credit-limit” test, there is a success route that shows what will happen when the test has expected inputs, and a failure route that shows what will happen if inputs are out of bounds or error causing.

The successful route input and output for the “add-credit-limit” test shows a credit of \$1000 being added in one session and approved, see as follows:

INPUT

```
username: "Buyer";  
user_type: BS;  
starting_credit: 100.00;  
session_credit_already_added: 0.00;  
first_credit_add: 600.00;  
second_credit_add: 400.00;
```

OUTPUT

```
first_addition: approved;  
second_addition: approved;  
session_credit_added: 1000.00;  
buyer_credit: 1100.00;  
final_session: null;  
records.length: 3;
```

The failure route input and output for the “add-credit-limit” test shows an added credit for a user exceeding \$1000 in one session by one cent, resulting in a failure, see as follows:

INPUT

```
username: "Buyer";  
user_type: BS;  
starting_credit: 100.00;  
session_credit_already_added: 0.00;  
first_credit_add: 600.00;  
second_credit_add: 400.00;  
third_credit_add: 0.01;
```

OUTPUT

```
first_addition: approved;
```

```
second_addition: approved;  
third_addition: rejected;  
warning: "Session credit limit exceeded.";  
session_credit_added: 1000.00;  
buyer_credit: 1100.00;  
final_session: null;  
records.length: 3;
```

This process will be automated using shell scripts (.bat file, as this is developed in a windows environment) rather than having each test manually performed each time a change is made to the program. We are implementing these tests using eXtreme Programming practices, hence why we started the process by creating these tests.

Output of the tests will be stored in an "output" directory folder with specific error codes that help with logging and debugging. Each test run will be logged with a new name (Run_0001, Run_0002) in order to easily identify most recent runs and not run into file name conflicts, and to allow previous runs to be retained to identify differences in output between program versions. The output files will be stored as plain text. The test directory will also contain a text file with all test inputs, descriptions, and expected outputs to allow for easier debugging and problem visibility.

Therefore, the test process will involve:

1. Creation/modification of feature/component.
2. Identification of possible errors or problems.
3. Test cases created for said expected errors.
4. Implementation of test cases.
5. Test cases added to pytest.
6. Pytest ran to identify any errors.
7. Repeat upon further modification.